

**LAPORAN TAKE HOME TASK
PRAKTIKUM PEMROGRAMAN**

BERORIENTASI OBJEK

MODUL 10



Disusun Oleh:

Ahmad Zaidan 105224044

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA**

2026

I. Pendahuluan

Program Sistem Gudang ini adalah implementasi bahasa pemrograman Java yang bertujuan pada konsep *Object-Oriented Programming* (OOP) lanjutan dan struktur data bawaan (*Java Collections Framework*). Di dalam materi ini terdapat penerapan enkapsulasi data melalui modifier privat pada atribut barang, serta penggunaan objek koleksi seperti HashMap untuk basis data pencarian barang, HashSet untuk menyaring kategori unik, dan ArrayList sebagai penampung log transaksi. Program ini dibuat untuk mensimulasikan sistem manajemen inventaris gudang secara digital guna mempermudah proses kontrol barang masuk dan keluar. Dengan demikian, pendahuluan modul ini bertujuan untuk memberikan pemahaman dasar bagaimana relasi antar-kelas dan manipulasi koleksi objek bekerja dalam menangani studi kasus pengelolaan stok barang.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	idBarang	String	Menyimpan kode unik identitas barang
2	NamaBarang	String	Menyimpan nama dari barang
3	kategori	String	Menyimpan pengelompokan jenis barang.
4	stok	int	Menyimpan jumlah kuantitas barang yang tersedia.
5	databaseBarang	Map<String, Barang>	Struktur data berupa Key-Value (ID Barang sebagai Key, dan Objek Barang sebagai Value) untuk menyimpan seluruh data barang.
6	kategoriUnik	Set<String>	Struktur data untuk menyimpan daftar kategori tanpa adanya duplikasi.
7	riwayatAktivitas	List<String>	Struktur data berurutan untuk mencatat log atau histori dari setiap transaksi/aktivitas di gudang.
8	gudang	SistemGudang	Objek utama yang digunakan untuk menjalankan dan mengakses fitur-fitur manajemen gudang.

III. Constructor dan method

No	Nama Metode	Jenis metode	Fungsi
1	Barang()	Constructor	Menginisialisasi nilai awal objek Barang (ID, Nama, Kategori, Stok) ketika pertama kali dibuat.
2	getidBarang()	Functional	Mengambil dan mengembalikan nilai ID Barang yang bersifat <i>private</i> .
3	getNamaBarang()	Functional	Mengambil dan mengembalikan nama barang yang bersifat <i>private</i> .
4	getKategori()	Functional	Mengambil dan mengembalikan kategori barang yang bersifat <i>private</i> .
5	getStok()	Functional	Mengambil dan mengembalikan jumlah stok barang saat ini.
6	setStok(int)	Procedural	Mengubah atau memperbarui nilai jumlah stok barang dari luar class.
7	SistemGudang()	Constructor	Menginisialisasi objek koleksi (HashMap, HashSet, ArrayList) saat sistem gudang dibuat.
8	tambahBarang()	Procedural	Memproses pendaftaran barang baru ke dalam database dan mencatatnya ke riwayat.
9	tambahStok()	Procedural	Menambah jumlah stok barang yang sudah terdaftar berdasarkan ID-nya.
10	kurangiStok()	Procedural	Mengurangi jumlah stok barang setelah memastikan stok mencukupi dan ID-nya valid.
11	cetakLaporan()	Procedural	Menampilkan ringkasan kategori, sisa stok, dan seluruh riwayat transaksi ke konsol.
12	Main()	Procedural	Pintu utama untuk mengeksekusi dan menjalankan seluruh simulasi program gudang.

IV. Dokumentasi dan Pembahasan Code

a. Barang.java

```
1 class Barang {
2     private String idBarang;
3     private String NamaBarang;
4     private String kategori;
5     private int stok;
6
7     public Barang(String idBarang, String NamaBarang, String kategori, int stok){
8         this.idBarang = idBarang;
9         this.NamaBarang = NamaBarang;
10        this.kategori = kategori;
11        this.stok = stok;
12    }
13
14    public String getIdBarang(){
15        return idBarang;
16    }
17    public String getNamaBarang(){
18        return NamaBarang;
19    }
20    public String getKategori(){
21        return kategori;
22    }
23    public int getStok(){
24        return stok;
25    }
26    public void setStok(int stok){
27        this.stok = stok;
28    }
29 }
30
```

→Bagian “class Barang”: Seluruh atribut internal barang (idBarang, NamaBarang, kategori, dan stok) dideklarasikan menggunakan akses modifier private. Hal ini bertujuan untuk menerapkan konsep *Encapsulation* (penyembunyian data atau *data hiding*) agar atribut-atribut tersebut tidak dapat dimodifikasi atau dibaca secara acak oleh kelas eksternal tanpa melalui prosedur kendali resmi.

→Bagian “public Barang”: Di bagian ini ada konstruktor public Barang(...) yang gunanya mirip kayak “pabrik” buat bikin objek barang baru. Jadi pas kita mau input barang ke memori, konstruktor inilah yang bertugas menangkap data-data mentah (seperti ID, Nama, Kategori, dan Stok) buat langsung dipasang ke objeknya. Di sini kita juga pakai kata kunci this (contohnya this.idBarang = idBarang;). Ini penting banget biar program gak bingung (kenal istilah *variable shadowing*). Fungsinya buat bedain secara jelas mana variabel milik kelas global yang disimpan di memori objek, dan mana variabel lokal yang baru masuk dari parameter.

→Bagian Getter: Kumpulan method getIdBarang(), getNamaBarang(), getKategori(), dan getStok() ini biasa disebut sebagai **Getter** (tipe *Functional*). Sesuai namanya, fungsinya cuma buat “ngambil” atau ngintip data aja alias *read-only*. Karena mereka wajib pakai perintah return, kelas lain (kayak SistemGudang) bisa tahu info detail barangnya tanpa ada risiko merusak atau mengubah data asli di dalamnya.

→Bagian Setter: Kebalikannya, method setStok(int stok) disebut sebagai **Setter** (tipe *Procedural* atau void). Method ini gak mengembalikan nilai apa-apa, tapi tugasnya fokus buat mengubah data.

Ini ibarat pintu gerbang resmi kalau ada bongkar muat di gudang; jadi kalau stok barang mau ditambah atau dikurangi, nilainya harus di-update lewat method ini biar tetap aman dan rapi.

b. SistemGudang.java

```
1 import java.util.*;
2
3 class SistemGudang {
4     private Map<String, Barang> databaseBarang;
5     private Set<String> kategoriUnik;
6     private List<String> riwayatAktivitas;
7
8     public SistemGudang(){
9         this.databaseBarang = new HashMap<>();
10        this.kategoriUnik = new HashSet<>();
11        this.riwayatAktivitas = new ArrayList<>();
12    }
13
14    public void tambahBarang(String idBarang, String NamaBarang, String kategori, int stok){
15        if (databaseBarang.containsKey(idBarang)){
16            Barang barang = new Barang(idBarang, NamaBarang, kategori, stok);
17            databaseBarang.put(idBarang, barang);
18            kategoriUnik.add(kategori);
19            riwayatAktivitas.add("Menambah barang dengan ID: " + idBarang);
20        }else{
21            riwayatAktivitas.add("gagal daftar: " + idBarang + " sudah ada");
22        }
23    }
24
25    public void tambahStok (String idBarang, int jumlah){
26        Barang barang = databaseBarang.get(idBarang);
27        if (barang != null) {
28            barang.setStok(barang.getStok() + jumlah);
29            riwayatAktivitas.add("Barang Masuk: " + idBarang + " ditambah sebanyak " + jumlah + " unit");
30        }else{
31            riwayatAktivitas.add("gagal nambah stok: " + idBarang + " tidak ada");
32        }
33    }
34
35    public void kurangiStok(String idBarang, int jumlah){
36        Barang barang = databaseBarang.get(idBarang);
37        if (barang == null){
38            riwayatAktivitas.add("Gagal Kurang Stok: " + idBarang + " tidak ada");
39            System.out.println("Gagal: " + idBarang + " tidak ada");
40            return;
41        }
42        if (barang.getStok() >= jumlah){
43            barang.setStok(barang.getStok() - jumlah);
44            riwayatAktivitas.add("Barang Keluar: " + idBarang + " dikurangi " + jumlah + " unit");
45        }else{
46            riwayatAktivitas.add("Kurang Stok: " + idBarang + " stok tidak cukup (Sisa: " + barang.getStok() + ", barang diminta: " + jumlah + ")");
47            System.out.println("Gagal: Stok " + idBarang + " tidak cukup.");
48        }
49    }
50
51    public void cetakLaporan(){
52        System.out.println(x: "DAFTAR KATEGORI UNIK");
53        for (String kat : kategoriUnik) {
54            System.out.println("- " + kat);
55        }
56
57        System.out.println(x: "SISA STOK BARANG");
58        for (Barang b : databaseBarang.values()){
59            System.out.println(b.getIdBarang() + " - " + b.getNamaBarang() + " (" + b.getKategori() + ") : " + b.getStok() + " unit");
60        }
61
62        System.out.println(x: "RIWAYAT AKTIVITAS TRANSAKSI");
63        for (String riwayat : riwayatAktivitas){
64            System.out.println("- " + riwayat);
65        }
66    }
67 }
68
```

→Bagian “class SistemGudang”: Di awal kelas, kita menyiapkan tiga wadah kosong bertipe private untuk menyimpan data gudang. Ada databaseBarang yang pakai Map buat memasangkan ID Barang dengan objek barangnya biar nyarinya gampang. Ada kategoriUnik pakai Set supaya kalau ada kategori yang sama diinput berulang kali, sistem otomatis menyaringnya agar tidak duplikat. Terakhir, ada riwayatAktivitas pakai List yang fungsinya mirip buku catatan buat mencatat semua histori transaksi secara berurutan.

→Bagian “public SistemGudang”: Ini adalah konstruktor dari kelas SistemGudang. Tugasnya simpel, yaitu untuk "menghidupkan" atau memesan tempat di memori (new) untuk ketiga wadah yang sudah dideklarasikan di atas tadi (HashMap, HashSet, dan ArrayList). Jadi, pas objek gudang pertama kali dibuat, wadah-wadah ini sudah siap dipakai dan dalam kondisi kosong.

→Bagian metode tambahBarang : Method ini dipakai buat mendaftarkan barang baru ke dalam sistem gudang. Konsepnya bertipe *Procedural* (void).

→Bagian metode tambahStok: Ini method *Procedural* yang gunanya untuk mengurus pasokan barang masuk. Cara kerjanya, sistem bakal mencari data barang dulu di database berdasarkan ID-nya (databaseBarang.get(idBarang)). Kalau barangnya ketemu (tidak bernilai null), stok lamanya akan diambil dan ditambah dengan jumlah yang baru masuk lewat method setStok(), lalu log suksesnya dicatat. Kalau ID-nya ga ada di database, sistem langsung nulis log gagal ke riwayat.

→Bagian metode kurangiStok: Method *Procedural* ini bertugas menangani barang keluar atau penjualan. Logika filternya ada dua tahap:

Pertama (Baris 37–41): Cek apakah barangnya ada di gudang. Kalau gak ketemu (null), sistem langsung memunculkan pesan error di konsol, mencatatnya di riwayat, dan langsung berhenti (return).

Kedua (Baris 42–48): Kalau barangnya ada, sistem bakal cek lagi apakah stok yang tersisa cukup atau gak buat dikurangi. Kalau cukup, stok langsung dipotong dan log sukses dicatat. Tapi kalau stoknya kurang, transaksi digagalkan dan sistem bakal ngasih tahu info sisa stoknya lewat konsol dan riwayat.

→Bagian metode cetakLaporan: Terakhir, ini method *Procedural* yang fungsinya buat rekap data alias cetak laporan akhir ke layar konsol. Menggunakan perulangan (for-each), method ini bakal membongkar isi dari ketiga wadah kita tadi secara berurutan: mulai dari menampilkan semua daftar kategori unik yang terdaftar, sisa stok akhir dari setiap barang secara detail, sampai menampilkan seluruh baris riwayat aktivitas transaksi dari awal hingga akhir.

c. Main.java

```
1 public class Main {
2     Run | Debug | Windsurf: Refactor | Explain | Generate Javadoc | X
3     public static void main(String[] args) {
4         SistemGudang gudang = new SistemGudang();
5
6         gudang.tambahBarang(idBarang: "B11", NamaBarang: "Sapu", kategori: "Furnitur", stok: 15);
7         gudang.tambahBarang(idBarang: "B22", NamaBarang: "Rak", kategori: "Furnitur", stok: 7);
8         gudang.tambahBarang(idBarang: "B33", NamaBarang: "Lampu", kategori: "Elektronik", stok: 20);
9
10        gudang.tambahStok(idBarang: "B11", jumlah: 10);
11        gudang.kurangiStok(idBarang: "B33", jumlah: 15);
12        gudang.kurangiStok(idBarang: "B22", jumlah: 10);
13        gudang.cetakLaporan();
14    }
```

→Baris 1–2: Di sini kita membuat public class Main yang di dalamnya ada method public static void main(String[] args). Method ini bertindak sebagai pintu gerbang utama atau tombol *start* ketika program Java dijalankan. Tanpa adanya method main ini, semua kode sistem gudang yang sudah kita bikin sebelumnya gak bakal bisa dieksekusi.

→Baris 3: Di baris ini untuk membuat objek baru bernama gudang dari kelas SistemGudang dengan perintah new SistemGudang();. Langkah ini otomatis memicu konstruktor di kelas

SistemGudang untuk berjalan, sehingga aplikasi mulai menyiapkan dan mengosongkan tempat penyimpanan (database, kategori unik, dan riwayat transaksi) di dalam memori komputer.

→ **Baris 5–7:** Di blok ini, objek gudang mulai beraksi dengan memanggil method `tambahBarang(...)`. Kita memasukkan 3 jenis barang contoh ke dalam sistem, yaitu Sapu (Furnitur, 15 unit), Rak (Furnitur, 7 unit), dan Lampu (Elektronik, 20 unit). Proses ini dilakukan untuk mengisi database awal gudang sebelum melakukan transaksi lainnya.

→ simulasi mutasi stok

- > Baris 9: Memanggil `tambahStok` untuk barang B11 (Sapu) sebanyak 10 unit.
- > Baris 10: Memanggil `kurangiStok` untuk barang B33 (Lampu) sebanyak 15 unit.
- > Baris 11: Memanggil `kurangiStok` untuk barang B22 (Rak) sebanyak 10 unit. *FYI*, karena stok awal Rak cuma ada 7 unit dan diminta keluar 10 unit, baris ini bakal memicu logika error "stok tidak cukup" yang sudah kita buat di kelas `SistemGudang`.

→ eksekusi cetak laporan

Baris 12: Sebagai penutup alur program, kita memanggil method `gudang.cetakLaporan()`. Method ini bertugas merangkum semua hasil akhir dari aktivitas di atas dan menampilkannya secara rapi di konsol, mulai dari daftar kategori unik yang tersaring, sisa stok akhir tiap barang, hingga rekaman log riwayat transaksi dari awal sampai akhir.

V. Kesimpulan

Program Sistem Gudang ini berhasil membuktikan bagaimana konsep enkapsulasi dan *Java Collections Framework* dapat digunakan untuk membangun sistem manajemen inventaris yang aman dan tertata. Melalui simulasi yang dijalankan, program mampu melakukan validasi mutasi stok secara otomatis—seperti menambah stok barang masuk, memotong jumlah stok barang keluar, serta menolak transaksi secara sistematis jika barang tidak ditemukan atau sisa stok tidak mencukupi permintaan. Catatan riwayat transaksi juga berhasil terekam secara kronologis berurutan bersamaan dengan penyaringan kategori unik tanpa adanya duplikasi data. Dengan demikian, program ini menunjukkan bahwa kombinasi pembungkusan data privat dengan struktur data koleksi yang tepat mampu menghasilkan sistem pelaporan pergudangan yang informatif, akurat, dan terhindar dari risiko manipulasi data ilegal dari luar kelas.

IV. Daftar Pustaka

Tim Asisten Laboratorium Komputer. (2026). Modul Praktikum Pemrograman Berorientasi Objek - Modul 10: Java Collection Framework. Program Studi Ilmu Komputer, Universitas Pertamina

Dosen PBO. (2026). Java Collection Framework. Universitas Pertamina